

Silent by Construction: A Casebook of Control-Plane Failures in a Production LLM Agent Fleet

Anton Dziatkovskii 

Platinum Software Development Company, Brisbane, Australia
dzyatkovskiy.a2@gmail.com · ORCID 0000-0001-7408-3054
<https://github.com/tonydz> · <https://tonydz.github.io>

September 2026

Abstract

The agent literature evaluates models. Production agent systems are stopped by something else: the *control plane* — the schedulers, registries, dispatchers and watchdogs that decide when and whether an agent runs at all. We report eleven dated failures of that layer from a multi-machine LLM agent fleet (Windows, macOS, a cloud anchor) running dozens of scheduled agent routines in production since June 2026. Each case is given as symptom → root cause → detection guard → what the system did before that guard existed, and each is anchored to a public artifact: a `stdlib`-only deterministic model of the mechanism in an open casebook repository, an upstream bug report, or a dated journal line naming the instrument that measured it. The unifying finding is a property, not a bug: these failures are *silent by construction*. A scheduler that loads zero tasks does not crash; a registry that gained a second top-level list returns a confident zero; a watchdog that accepts any reply as service reports “0 orphans” for 22 consecutive runs across two real orphans; a detector with no executor raises a true alarm nobody can act on. Of 760 dated breakage lines in our journal the largest family is a routine that stopped and could not be auto-healed (37 lines), and a fleet audit found that of 243 enabled scheduled tasks only 41 — 17% — could name an output distinguishing “it ran” from “work landed.” We distill five recurring mechanisms — all-or-nothing validation, writer/reader contract drift, the defaulting accessor, the liveness proxy that measures the guard instead of the guarded, and proof-by-intention — and the cheap invariants that turn each from silence into an alarm. This is an experience report over one fleet ($N=1$), not a benchmark; we say plainly which cases are externally reproducible today and which are still only ours.

1 Introduction

An LLM agent that is running can be evaluated. An LLM agent that never started cannot, and in production that is the more common condition.

Beneath every deployed agent there is a layer of unglamorous machinery that decides *when* and *whether* it runs: an OS scheduler or cron, a registry file listing the routines, a dispatcher that spawns sessions, health checks that claim the whole arrangement is alive. We call this the control plane, borrowing the term from networking, because the analogy holds: it carries no payload, and when it fails the payload simply stops moving.

This layer fails differently from the models above it. Model failures are loud and legible — a wrong answer, a refusal, a traceback. Control-plane failures are *silent by construction*. A registry loader that rejects a malformed entry by refusing the whole file raises nothing an operator sees; it loads zero tasks, and zero tasks is a valid state. A watchdog that accepts any reply as evidence of health does not lie: it answers the question it was asked, which was the wrong question. In every case below the machinery kept running, the dashboards stayed green, and the work stopped.

We have run a multi-machine agent fleet in production since June 2026: an always-on hub, two laptops, a collaborator’s machine and a cloud anchor, coordinating over a synced file bus and running dozens of scheduled agent routines. The architecture and its consensus layer are a companion experience report [2]; this paper is the part of that experience the architecture did not prevent, filtered hard — a case appears only with a date and a public artifact.

Our contributions are those of a casebook: eleven dated control-plane failures with root causes and the guards they produced, grouped by the question the layer was answering — what is scheduled (§5), what gets launched (§6), whether it worked (§7); an explicit evidence tiering (§3) separating the three cases a stranger can reproduce today from those resting on our own instruments; the aggregate shape of the corpus (§4), including the measurement that our own class counter was miscounting, which changes how those aggregates should be read; and five recurring mechanisms with the cheap invariants that detect them (§8).

We make no novelty claim. Every guard here is something a careful engineer would recognize; that is the point. The contribution is the documented cost of not having them, in a regime — unattended agents spawning unattended agents — where the cost is paid in silence.

2 What the control plane is, and why it goes quiet

By control plane we mean everything between “a routine exists” and “an agent is running it”: (i) the *registry* — the file or store listing routines, their schedules and their enabled/disabled state; (ii) the *loader* that reads the registry into the scheduler; (iii) the *dispatcher* that spawns a session when a slot comes due, including the catch-up path after a restart; (iv) the *watchdogs* that assert the arrangement is alive; and (v) the *ledgers* where completion is recorded and read back.

Three structural properties make this layer quiet. First, **absence has no error code**: the observable of a working scheduler is an event that happens, and its failure observable is an event that does not. Nothing is positioned to notice a non-event unless something explicitly counts expected occurrences — exactly the instrumentation written last. Second, **every convenient liveness proxy measures the wrong thing**: process running, heartbeat replying, log growing, task list rendering. Each is cheap to check, and each stays true through all eleven failures below; in one case the scheduler kept *writing* bookkeeping fields into the very registry file it could no longer load. Third, **the unattended agent has no operator**: a human sees an empty output and frowns, while a routine at 03:20 receives an empty list, writes “0 items processed,” exits 0, and the report flows into the next decision. The zero is not an error; it is an answer, in the same format as the truth.

Related work, honestly. We know of no literature on this layer, and believe the absence is real rather than a gap in our reading. Multi-agent LLM frameworks [8, 9, 10] orchestrate roles inside a process or a service; what launches the process is out of scope by construction, and in a benchmark harness that is a test runner, not a scheduler surviving restarts, account switches and week-long unattended operation. The operational side is instead covered by two mature bodies of practice that do not meet here: site reliability engineering, which has long known that alerting on symptoms beats alerting on causes and that a monitor’s own health is a first-class concern, and the folklore of cron — encoding, environment and catch-up bugs any operations engineer recognizes on sight. We claim neither that these mechanisms are new nor that we found them first, only that unattended agents spawning unattended agents raises the price of each, because the observable is a session that never started and no operator is watching the terminal it never printed to. What we add is a dated, artifact-backed record of that price.

3 Method and evidence tiers

Two rules governed inclusion. First, **date and artifact**: a case is admitted only with a calendar date and a public artifact — a case directory in the open casebook repository [1], an upstream issue on a public tracker,

or a journal line naming the instrument and the file it wrote. Cases we remember but cannot point at were dropped. Second, **mechanism, not configuration**: failures are abstracted to registry shapes, loader contracts and timing, never to our private paths, hosts or data.

Because “we have evidence” is itself a claim of varying strength, we tier it explicitly and mark every case below:

Tier A — externally reproducible (C1, C2, C5): the mechanism ships as a stdlib-only, fixed-clock Python model in [1] together with a detection guard whose test is *shown red* against the broken behavior — a runnable `-against-broken-*` mode, not a claim in a README. A stranger with Python 3.8 and no network can run it. **Tier B — publicly filed, not packaged** (C3, C6): a dated production incident written up on a public bug tracker with environment and log excerpt, but no published repro model. **Tier C — our instruments only** (C4, C7–C11): a dated journal line, the measurement, and the instrument that produced it; the instrument is ours and the reader takes the number on our word.

A note on what a repro is and is not, which the casebook states once and repeats per case: the models reproduce the *mechanism* — the load contract and its failure shape — deterministically. They are not the vendor’s code and do not reproduce a specific build’s internals. Real-world evidence is cited separately so the mechanism can be checked against it.

4 The corpus in aggregate

The fleet keeps an append-only incident journal: one breakage, one line, four fields (symptom, conditions, components involved, hypothesized cause), written through a single entry point to per-machine shards and folded into a shared view. As of 2026-09-10 it holds 760 dated lines under 539 distinct class names; the control-plane families are the largest single block (Table 1).

Family (class name)	Lines	Character
dead-scheduled-task	37	a routine stopped; auto-heal had no runbook for it
canon-drift-node-silent	28	a node stopped acknowledging shared-rule updates
browser-rail-down	19	the browser automation rail died under a scheduled run
run-timeout-or-hang	17	an unattended run hung rather than failed
rate-limit-loses-the-day	13	the subscription bucket emptied; scheduled work skipped
instrument-lies-green	10	the measuring device reported health it had not measured
silent-green-watchdog	8	the guard was alive; the guarded thing was dead

Table 1: Largest breakage families in the fleet journal, 2026-08-10 to 2026-09-10.

These aggregates come with a caveat we would rather state than bury, because it is itself a control-plane failure of the reporting layer. On 2026-08-22 an audit found 722 records under 648 distinct names, 97% of them singletons, with only 7 classes reaching the “three dated occurrences” threshold that triggers a systematic repair in our process. The cause was mechanical: the tool assigning a class compared names *exactly*, with no synonym map, so one defect recorded on three days under three phrasings counted as three unrelated singletons — a counter producing numbers without producing knowledge. Counting by *family* instead moved `silent-green-watchdog` from 8 lines to 58 dated cases between 2026-08-10 and 2026-09-04 under roughly twenty names. Read Table 1 as a lower bound per family, not as a partition.

5 Group I: what is scheduled

The first three cases share a shape: the registry on disk and the registry in the scheduler’s memory disagree, and nothing compares them.

C1 — One malformed field kills the whole registry. (Tier A; 2026-08; upstream #90533 [7].) *Symptom.* Every scheduled agent routine on a Windows node stopped firing at once: 35 tasks dark for 55 hours. Nothing on screen changed, processes ran, the dispatcher heartbeat replied. *Root cause.* Two cooperating defects, both required. First, writer/reader contract drift *inside one product*: the platform’s own task-update tool documented and wrote the next-fire timestamp as an ISO 8601 string, while the registry loader validated that field as epoch milliseconds — the write path produced data the read path rejected. Second, all-or-nothing validation: on the first invalid entry the loader rejected the *entire* registry rather than quarantining the one bad record. One bad field out of 35 tasks → zero tasks loaded. *Guard.* Registry parity: count task definitions on disk, count tasks the scheduler actually loaded, alarm on mismatch. Cheap, deterministic, and it also covers registry wipes and the split stores of C3. *Without the guard.* 55 hours of total, invisible outage, ended not by the platform and not by any process check but by an external check on the *age of the output* of one routine. *Residual risk.* A running app holds the registry in memory: hand-editing the file is inert until restart, and nothing on screen says so.

C2 — A second top-level list makes readers report a confident zero. (Tier A; 2026-08; upstream #82056 [7].) *Symptom.* A control-plane reader answered **0** for an entry that was registered. No exception, no warning — a number, in the format of the true answer. The zero came within a second of being written into the incident journal as a finding. *Root cause.* The registry grew a second top-level list, and the entries this reader wanted moved under the new key — so its own key was now *absent*. The reader was written as `d.get(k, d)`: with the key gone, the default returned the dictionary itself, and the scan then ran over the dictionary’s *keys*, a sequence of strings that will never match an entry. No exception, no warning, a count of zero. Written as `d[k]` the same line would have raised. *Guard.* Two rules, both one line. Never print a zero without the size of the population searched beside it. Never let an accessor default to an iterable of the wrong type. *Without the guard.* A false zero is worse than a crash and cheaper to produce: a crash is fixed in a minute, while a zero propagates into the report and into the next decision. In our case it nearly became a journal entry asserting the opposite of the truth.

C3 — Split-brain registries: the routine is on disk and not in the schedule. (Tier B; 2026-08; upstream #89840 [7].) *Symptom.* Routines invisible after an account switch. Measured on one node: 269 task prompts present on disk, 23 registered in the live schedule. *Root cause.* The store is partitioned per account and per workspace, so “the registry” is not one object; a session created under one identity writes to a store that a session under another identity never reads. The filesystem view and the scheduling view answer different questions and look identical to an operator. *Guard.* The same parity count as C1, but aggregated across *all* stores on the node rather than the current one — a per-account count is a proxy that stays green through exactly this failure. *Without the guard.* An `ls` of the task directory is a confirmation ritual that confirms nothing: the folder is the intention, the registry is the fact.

6 Group II: what actually gets launched

The dispatch path obeys different rules than the schedule, and every one of these cases lives in that gap.

C4 — Sync carries the folder, not the registration. (Tier C; 2026-09-10.) *Symptom.* A daily routine was believed to be running on three nodes. It was running on one. On the hub the task’s directory was present, synced, and correct; the routine had never fired. *Root cause.* Two stores with different replication. The task’s definition directory replicates with the rest of the configuration and therefore appears everywhere; the live schedule lives in a node-local registry belonging to the desktop application’s account and does not replicate at all. Registration must be performed by a live session *on that node*. The folder’s presence is a false green with an obvious-looking check behind it. *Guard.* A status instrument reading the node-local registry directly

and reporting, per node: registered, enabled, last run. The registry keys entries by id, not by the taskId a caller passes, and our first version searched the wrong key and confidently reported “absent everywhere” — including on the node where the routine was demonstrably alive. *Without the guard.* A routine believed live on the hub, with a directory listing as the evidence. A second-opinion review of the instrument found two further defects before it shipped: an `enabled:false` entry counted as a live routine, and a truncated registry file reported as “not registered” rather than “I do not know” — now distinct exit codes, each closed by a test shown red.

C5 — A restart re-fires already-run and disabled tasks, invisibly. (Tier A; 2026-07-03; upstream #74055 [7].) *Symptom.* After a scheduler restart at 23:33, six daily tasks whose cron slots were between 00:20 and 05:40 fired a second time, within minutes of each other, ten hours late. Separately in the same episode, a task explicitly disabled three days earlier fired at 20:29, and a Monday-only weekly task fired on the wrong weekday. These are full agent sessions with cold context loads: per the upstream write-up, whose token figures were read off the desktop app’s own session logs, one nightly task accounted for roughly 24.8M tokens across its two runs. *Root cause.* Three cooperating defects. Restart catch-up re-fires any task whose slot is in the past for the current period without checking whether it already ran in that period. The disabled flag is honored for the schedule but not re-checked at fire time on the catch-up path. And catch-up fires do not write the last-run timestamp — so the one field an operator would inspect stays stale. *Guard.* An idempotency key per (task, period), re-checking enabled at fire time, and writing the run stamp on *every* dispatch including catch-up. *Without the guard.* Duplicate unattended sessions consuming tens of millions of tokens while the registry claimed the tasks had not run. This is the case that taught us the guard test must assert the *dispatch list*, not the stamp: an assertion like “the disabled task has no last-run timestamp” passes on the bug, because the absence of the stamp *is* the bug. *Residual risk.* The verified operator workaround — moving the task’s folder out of the scheduled set — is destructive to the task, and disabling alone does not stop the ghost fires.

C6 — Encoding kills the cron, and only the cron. (Tier B; 2026-08; upstream #90962 [7].) *Symptom.* A script written by an agent ran correctly when launched by hand and died when launched by the scheduler, producing no log at all. *Root cause.* The file was written without a byte-order mark. Windows PowerShell 5.1 decodes a BOM-less script using the system ANSI code page, so the non-ASCII content in it was mis-decoded and parsing failed before the script’s own first logging line could execute — a failure with no output by construction. Why the hand-run succeeded and the scheduled run did not is the part we observed rather than explained: the two invocations differed in effective code page, and we did not isolate which of the several differences between an attended console and the scheduler’s host process was decisive. *Guard.* Write agent-generated scripts with an explicit encoding contract (BOM where the platform’s scheduled path requires it) and treat “runs by hand, dies on schedule” as a distinct diagnosis with encoding as its first hypothesis. *Without the guard.* A routine that is dead only in production and healthy in every reproduction attempt an operator makes.

C7 — Editing a batch file while it is executing. (Tier C; 2026-09-10.) *Symptom.* A nightly pipeline reported a scoring step as failed with exit code 9009 — immediately after that same step had printed a perfect result — and then re-executed a later step. *Root cause.* The command interpreter reads a batch file by *byte offset*, line by line, as it executes. Comments added to the file while it was running shifted the offset; the interpreter’s next read landed in the middle of a line, produced a “not recognized” error from the fragment, and that error’s code was captured as the step’s exit status. The deeper cause is that step orchestration lived inside the batch file itself, which is therefore both the program and a live, editable file. *Guard.* Never edit a script while its scheduled run is in flight, and — the actual repair — move step orchestration out of the shell

script so that the file being edited is not the thing being executed. *Without the guard.* A false failure verdict on a step that succeeded, and a duplicated execution of the following step. Note the shape: the pipeline did not crash, it *lied*, and the lie was internally consistent.

7 Group III: whether it worked

The last group is the most uncomfortable, because these are the failures of the machinery we built specifically to catch the earlier ones.

C8 — Green means the code reached the report line. (Tier C; 2026-09-04.) *Symptom.* Everything was green while work was not landing. *Root cause.* A fleet-wide audit compared enabled scheduled tasks against routines that could name an *output* — an artifact whose freshness an outsider could check. Result: **243 enabled tasks, 41 with a declared output (17%)**. For the remaining 202 the green signal meant “the task fired,” not “the work landed.” We are careful about what this number says: it is not “202 routines are broken,” it is “for 202 routines we had no way to tell a working run from an idle one.” *Guard.* A ratchet on the default: every *new* routine must, at birth, either name an output artifact in a freshness registry or write a line with a reason into an exemption file; the pre-existing debt is snapshotted and stays quiet, and only new unnamed routines raise an error. This was chosen over the two obvious alternatives — a twelfth watchdog, or a one-off cleanup of 202 rows — because both leave the default unchanged. *Without the guard.* 83% of the fleet’s scheduled work had no observable that distinguished success from silence. *On the instrument itself.* It runs known-answer fixtures before every production pass and, on disagreement, exits with a distinct code and says *nothing* about production data — adopted after a related instrument reported a plausible number from a degenerate input. Its own runs then exposed two more defects: it reprinted a character-for-character identical alarm block on four consecutive nights (a detector repeating itself verbatim stops being read), and healed items never left its snapshot. Both were closed by a silence ladder suppressing the *printing* while keeping the non-zero exit code, proved against a kill-list of eight mutations, each shown red.

C9 — The watchdog that accepts any reply as service. (Tier C; 2026-08/09.) *Symptom.* A health check reported “0 orphans” for 22 consecutive runs while two real orphans existed. A connector health routine reported an overall green in nine consecutive runs while one connector returned UNKNOWN in every one of the nine. *Root cause.* The check verified that the service *answered*, not that the answer meant health, and aggregation treated “unknown” as “not a failure.” Same family: a restart watchdog on a macOS node logged 1583 restarts over 8 days, 40 of them failed starts, without one line reaching a human. The watchdog was alive; the guarded process was dead within seconds of each start. The mechanism, isolated by measurement rather than inferred: the watchdog ran as a periodic launchd job and started the browser as a background child, so the child remained a member of that job; when the job’s script exited, the remaining job members were terminated with it (measured: alive at +12 s, gone by +37 s). Relaunching through the OS launch service instead, so the process is not a member of the job, gave alive at +60 s. Note that this is a property of the job supervisor, not of parent exit in general — an orphaned child of an ordinary parent is simply reparented. Surfaced 16 days later, in a retrospective. *Guard.* A health check must assert a property of the *answer*, aggregation must treat “unknown” as red rather than as absence of red, and a watchdog must have a mouth: “restarted N times” is an alarm, not a routine event. *Without the guard.* Sixteen days of a dead rail behind a healthy monitor.

C10 — The detector with no executor, and the alarm nobody can close. (Tier C; 2026-08/09.) *Symptom.* True alarms, correctly raised, indefinitely. *Root cause.* Two variants of one defect — a verdict is a terminal state rather than a queue entry. (i) A canon-drift watchdog raised the *same* alarm 624 times in 7 days (626 runs total, 99.7% returning non-zero, growing an 8.8 MB log), while the deduplication that suppressed

the repeats converted “nobody is fixing this” into “quiet.” (ii) A connector alarm printed correctly every morning for 52 days with a repair tool that existed and had been verified, and the repair required a human hand that nothing forced. (iii) The largest family in the journal is exactly this shape: 37 dated lines reading “the self-healing robot could not heal this task,” each with the field `runbook: none` — the healer was fail-closed and correct, and had nothing to execute. *Guard.* Every alarm carries an owner, a deadline and a named next action; a repeat escalates rather than deduplicating; and one whose next action needs a human gets a forcing point — a place where the person collides with it — not a line in a log. *Without the guard.* A watchdog degrades into background noise, and the log it fills is the evidence that nobody read it. *A related trap.* A guard written weeks before it is wired enforces the rules *of its writing date*: ours, written 2026-08-01, blocked edits to a policy file above a size threshold that a 2026-08-21 amendment explicitly overrode. Diff a dormant guard’s date against the amendments made since.

C11 — Proof by intention: the ledger that was paid with a plan. (Tier C; 2026-09-10.) *Symptom.* The node’s debt ledger reported almost nothing outstanding — its command-line reader said `unpaid=2`, the node heartbeat printed `debt=1` — while dozens of items were in fact undone. *Root cause.* Completion was recorded as a free-text proof, and inspection found at least 28 of 663 records settled with text of the form “planned in . . .” — an intention accepted as evidence. Every consumer read those as paid, so the instruments honestly reported a green number derived from a corrupted definition of *paid*. *Guard.* Two halves, and the second is the one we would have skipped: a write-side gate rejecting intention-shaped proofs, *and* a read-side rule counting only non-plan settlements. A write gate alone would have left the historical false-greens invisible forever — an objection raised by an independent review panel before rollout. *Without the guard.* The same ledger, the same day, read under the corrected definition: `unpaid=2` → `unpaid=71`. The delta is larger than the 28 records we identified by hand because the read-side rule reclassifies every plan-shaped settlement, not only the ones inspection surfaced; 28 is a floor on the defect, not an accounting of the difference. *A sibling of the same family.* A component built specifically against the class “a debt is registered and no one executes it” was itself never wired into any node’s configuration: written 2026-07-17, referenced by zero callers for 49 days, during which the ledger it was meant to drain accumulated 193 unpaid items, the oldest 478 hours old. Worse, the verb its launcher would have invoked did not exist in it; the dispatcher’s `else` branch would have silently run different work. Acceptance of a component is three grep questions with string answers: who calls it, is the caller live, and does the exact command the caller sends exist inside it.

8 Five mechanisms

Across the eleven cases, five mechanisms recur. They are not deep; their depth is irrelevant to how much they cost.

1. **All-or-nothing validation on a collection.** One invalid element rejects the set (C1). In a registry fed by heterogeneous writers corruption is local; the reader must quarantine the element and load the rest. The same principle rescued a corrupted append-only log elsewhere in the fleet [2].
2. **Writer/reader contract drift, often inside one system.** The path writing a field and the path validating it disagree (C1), or the store’s shape changes and readers never learn (C2, C3, C4). Data a system wrote itself is not automatically data that system can read.
3. **The defaulting accessor.** `d.get(k, d)` invents an answer where `d[k]` would have raised (C2). Every default in a read path is a decision to fabricate rather than fail, and here the fabrication is usually zero.
4. **The liveness proxy that measures the guard, not the guarded.** Process alive, heartbeat replying, folder present, task fired (C4, C8, C9): properties of the machinery, none of the work. The repair has one shape — measure the *age of the output* at the consumer.

5. **Proof by intention.** A plan, a dispatch or a verdict accepted as completion (C10, C11). A proof is a fact — a command with its exit code, a file name, a before/after counter — never a statement of what will happen.

Two meta-observations tie these together. *The instrument is a claim too, and it expires*: here alone the class counter miscounted (§4), the freshness auditor carried two defects (C8), and the registry status tool searched the wrong key (C4) — every number above is downstream of a device that itself needs a control sample. And *a test that has never been red proves nothing*: every Tier-A guard ships with a runnable mode wiring its tests to the broken behavior, which must fail. We adopted this after finding guard tests that passed against the very bug they were written for (C5).

Guards that earned their place

What we now run. None requires a service, a database or a dependency; all are deterministic and make no model calls.

1. **Registry parity** — definitions on disk vs. tasks loaded, aggregated across every store on the node (C1, C3, C4, registry wipes).
2. **Output freshness at the consumer**, enforced by a ratchet at routine birth (C8). The only check here that survived every mode, because it measures the work.
3. **Idempotency key per (task, period)** plus a run stamp on *every* dispatch (C5).
4. **Fail on unknown** — aggregation treats “unknown” as red (C9’s nine green runs).
5. **Population printed beside every zero** (C2 and its whole family).
6. **Fact-shaped proofs, gated on write *and* on read** (C11; only the read side recovers history).
7. **Acceptance as three grep questions** — who calls it, is the caller live, does the exact command exist inside it (C11).
8. **A control sample of known-answer fixtures** before every production pass of an instrument.
9. **A silence ladder on printing only** (24h → 72h → weekly), exit code unchanged: suppress the noise, never the verdict (C8, C10).

9 Limitations

This is one fleet, $N=1$, operated by the author over roughly three months, with no control group, no baseline, and no blind assessment. The aggregates count our own journal lines, and §4 documents that the counter itself was defective for part of the period; treat family sizes as lower bounds. Only three cases (C1, C2, C5) are reproducible by a stranger today, and even those reproduce a *model of the mechanism*, not a vendor’s code path — stated deliberately rather than letting a deterministic script imply more than it demonstrates. Two (C3, C6) rest on public bug reports whose root causes the vendor has not confirmed. Six rest on our instruments, several of which we have shown to be fallible in this same paper.

We deliberately do not score, rank or compare platforms: a scoring harness earns the right to exist only after cases have external reproductions and at least one vendor-confirmed root cause, and ranking a layer nobody has characterized produces numbers, not knowledge. Nor can we claim any guard here prevents future failures — each was earned after an incident, which is evidence of a debt paid, not of safety. We would consider this paper usefully falsified by an external reproduction that fails, a vendor confirmation that a root cause lies elsewhere, or a fleet of comparable shape whose journal is dominated by different families. All three would be worth more to us than agreement.

Code, data and contact

The casebook is a live, open repository: <https://github.com/tonydz/agent-control-plane-casebook> (MIT). Each published case is a directory under `incidents/` with the symptom, root cause, a stdlib-only deterministic repro, a detection guard with its red-mode test, mitigation with residual risk, and dated evidence with upstream links. External reproductions — especially failed ones — are the most valuable contribution (CONTRIBUTING.md); corrections: open an issue.

Related open components from the same fleet: the consensus engine [3]; the gate deciding whether an unattended agent may interrupt a person (`agent-approval-gate`); the deploy tool that rolls a change to every machine and proves it landed (`fleet-deploy`); the independent panel that reviews what an agent built before it may say “done” (`secondop-panel`); the tooling that publishes internals without leaking them (`oss-publish`) — all under <https://github.com/tonydz>.

Author: Anton Dzyatkovskii — dzyatkovskiy.a2@gmail.com · ORCID 0000-0001-7408-3054 · <https://github.com/tonydz> · <https://tonydz.github.io>. Companion preprints: [2, 4, 5, 6].

Authorship, responsibility, independence. Case documentation and repository code are drafted by an AI agent operating as the author’s synthetic co-founder and published under the author’s editorial control; the author is the responsible human for every claim here and in the repository, every commit carries provenance trailers naming tool, model, machine and operator, and corrections are logged publicly. This work is not affiliated with, endorsed by, or reviewed by any vendor; upstream links point at public bug trackers, several of the referenced reports filed by this lab.

References

- [1] A. Dzyatkovskii. `agent-control-plane-casebook`: Reproducible control-plane failures from a production multi-agent fleet. <https://github.com/tonydz/agent-control-plane-casebook>, 2026. MIT license.
- [2] A. Dzyatkovskii. Operating a human-governed multi-machine LLM agent fleet: An experience report. Preprint, 2026. doi:10.5281/zenodo.22639714.
- [3] Palo Alto AI Research Lab. `claude-consensus`: A multi-machine consensus protocol for LLM agent fleets. <https://github.com/tonydz/claude-consensus>, 2026. MIT license.
- [4] A. Dzyatkovskii. Homeostatic governance: A Gorsky–Anokhin alternative to the Viable System Model for decentralized organizations. Preprint, 2026. doi:10.5281/zenodo.22639712.
- [5] A. Dzyatkovskii. Le Chatelier tokenomics: A thermodynamic–homeostatic framing of negative-feedback token supply control. Preprint, 2026. doi:10.5281/zenodo.22639710.
- [6] A. Dzyatkovskii. When does graph expansion help personal-corpus retrieval? A query-class-stratified evaluation protocol. Preprint, 2026. doi:10.5281/zenodo.22639718.
- [7] Public bug reports cited above, all on `anthropics/claude-code`, 2026, at <https://github.com/anthropics/claude-code/issues/N>; several were filed by this lab. #90533 one malformed `fireAt` rejects the entire task registry; #74055 a restart re-fires already-run and disabled tasks without recording `lastRunAt`; #82056 a consumer cannot tell a whole index from a truncated one from none; #89840 a task store partitioned per account and workspace hides routines after an identity switch; #90962 a BOM-less agent-written script parses as ANSI under the scheduler and dies without a log.
- [8] Q. Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.
- [9] S. Hong et al. MetaGPT: Meta programming for multi-agent collaborative framework. arXiv:2308.00352, 2023.
- [10] D. Gao et al. AgentScope: A flexible yet robust multi-agent platform. arXiv:2402.14034, 2024.